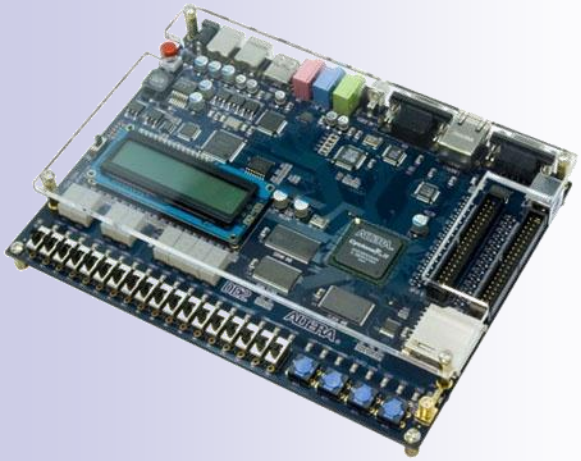
Logic Systems and Processors



Control Panel

CTU-FEE in Prague, CR – subject BE5B35LSP

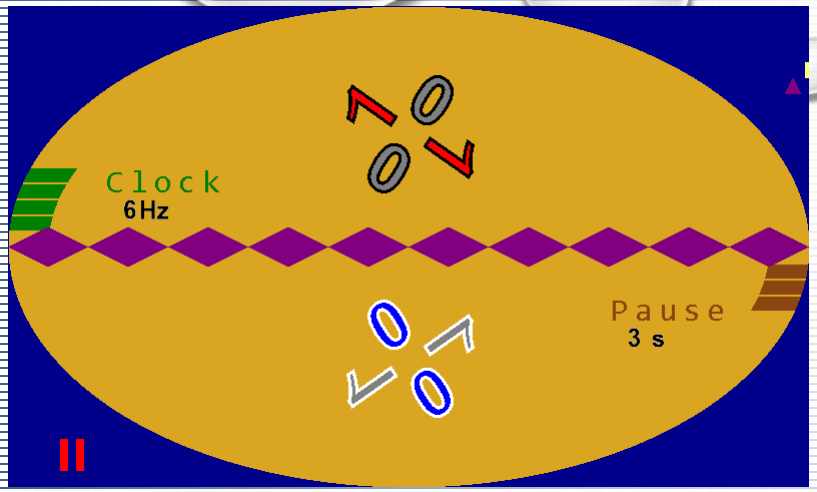
MAIN TASK LSP

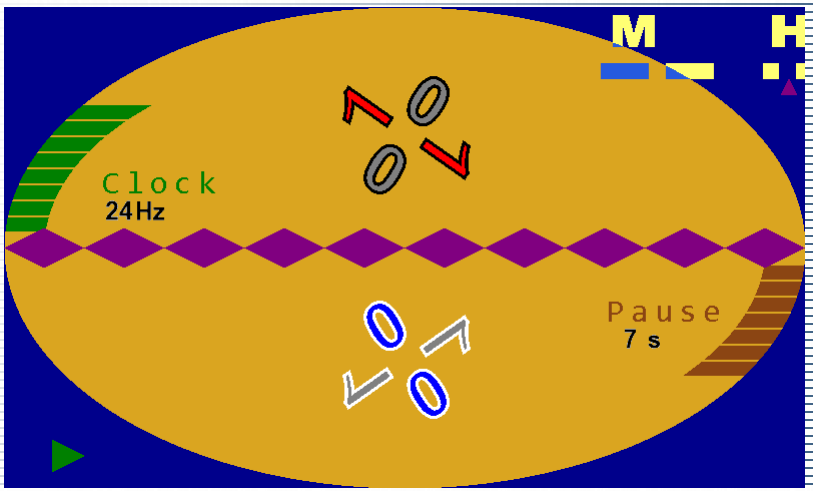
Control Panel of Morse Beacon — from 1 to 36 points

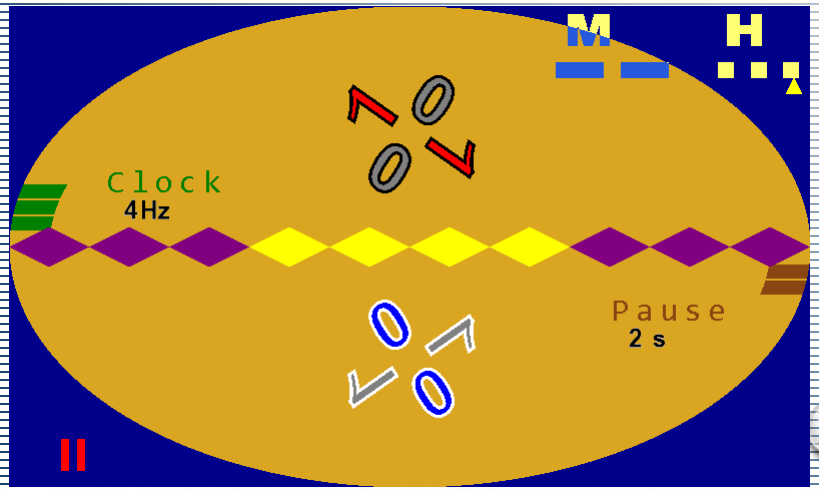
- Task_ControlPanel.pdf — requirements
- TouchIRDA_Base_2025w.zip — Quartus project version 2025w for winter semester (Sept 17, 2025). It demonstrates the use of data from a touch-sensitive LCD panel and the infrared remote control, and has a structure that allows simulation with multiple image recording.
- Demo_ControlPanel_36points_testbench80frames_plus_sof_file.zip — solved compiled task (*.sof) for your inspiration with testbench long 80 frames

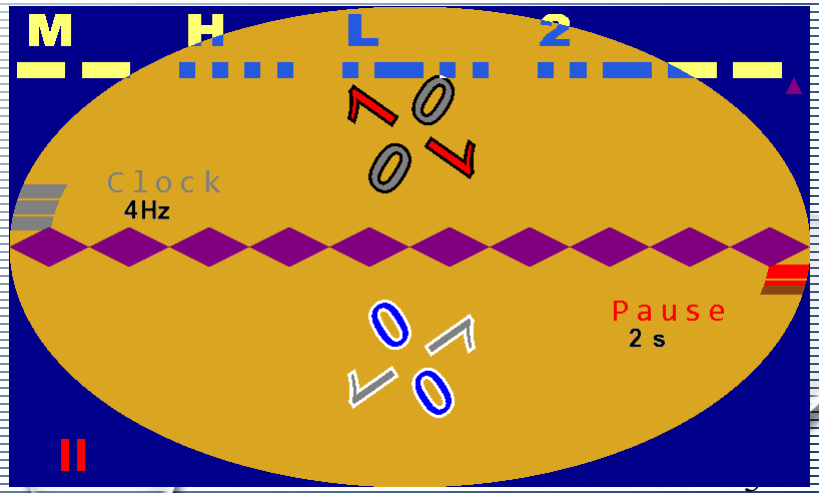
Note: The touch panel was assigned a year ago. Everyone solved it, and relatively quickly. But the time required to resolve the issue increased with the number of violations of the recommended conditions for circuit operation.

TouchIRDA_Base



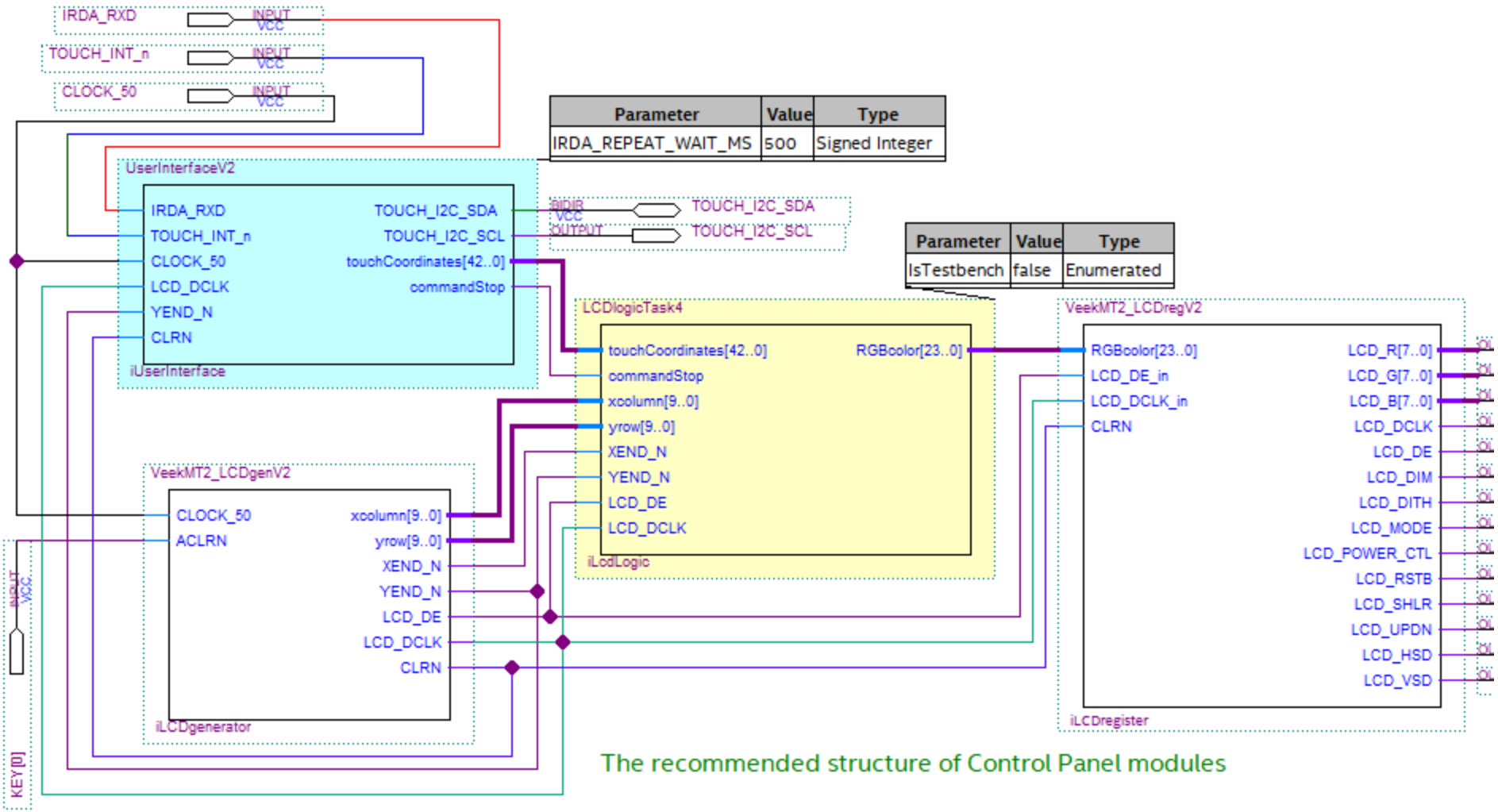






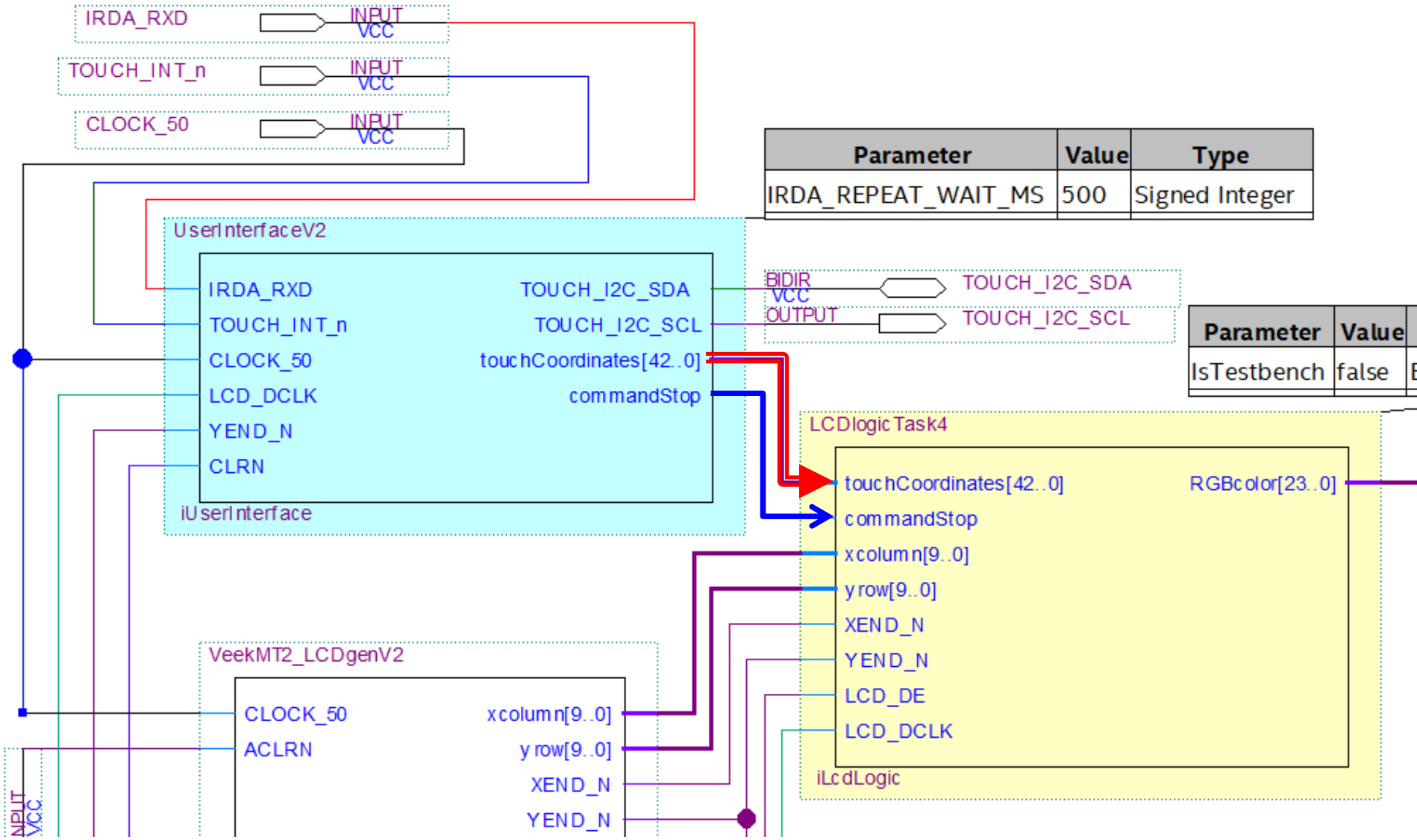
Demo_ControlPanel.bdf

from TouchIRDA_Base project



The recommended structure of Control Panel modules

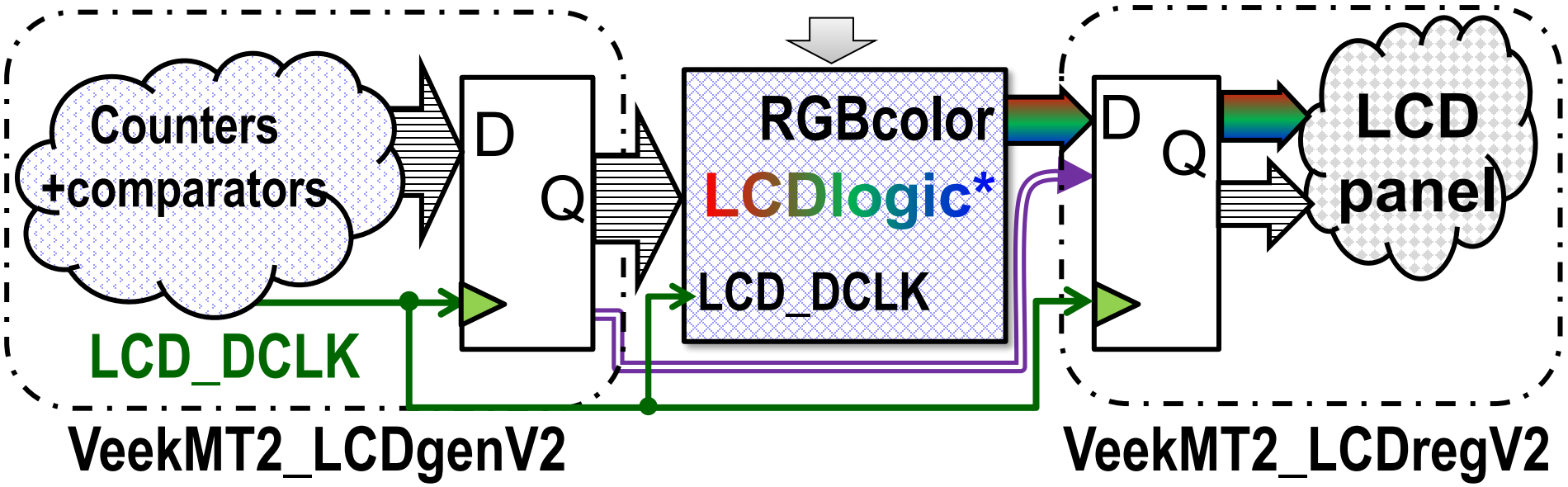
UserInterface



Parameter	Value	Type
IRDA_REPEAT_WAIT_MS	500	Signed Integer

Parameter	Value
IsTestbench	false

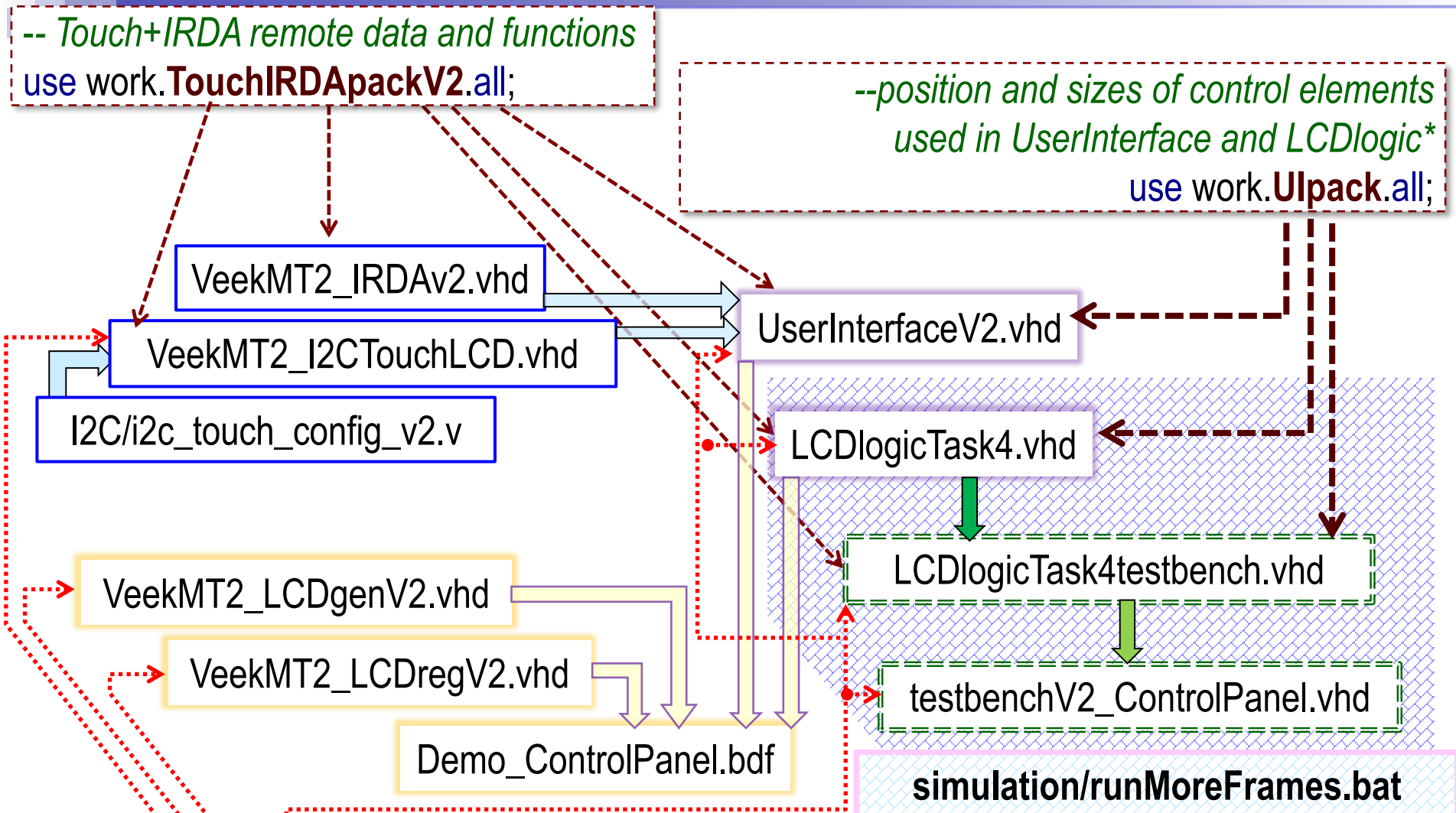
RGBcolor is the output of combinational logic!



LCDlogic lies between a pair of registers, and its combinational logic* ***deliberately*** *does not include a register for RGBcolor output. The colors assigned to x,y pixels are not delayed relative to other LCD synchronization signals. Only the LCD register loads them.*

Clocks

- LCDlogic* has an LCD_DCLK clock (**33 MHz**) connected to it, and XEND_N (**32.2 kHz)** and YEND_N (**61.4 Hz**) derived from it. By detecting their edges (rising_edge), we can create registers for auxiliary signals that influence RGBcolor values.
- But **RGBcolor itself must remain the output of combinational logic**. If the condition is violated, image distortions can occur. Many students tested :-).
- The touch panel and the infrared remote control cannot be simulated. The **UserInterface should send only values,** i.e., the results of IRDA and touch commands.
  We can simulate these values by VHDL code in LCDlogic4testbench, so GDHL simulations work for LCDlogic* .



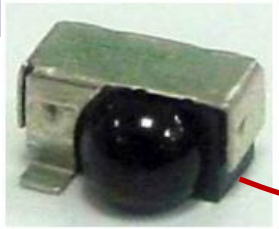
simulation/runMoreFrames.bat

Its compile list includes not only 3 *.vhd files in the frame above, but also all files used in them. All files are listed in their compile order, i.e., the packages are the first ones!

use work.**LCDpackV2**.all;

*-- LCD panel geometry and colors. It is used in all LCD related *.vhd codes.*

UserInterface



```
use work.LCDpackV2.all;
use work.TouchIRDApackV2.all;
use work.UIpack.all;
```

IRDA_RXD

entity work.VeekMT2_IRDAv2

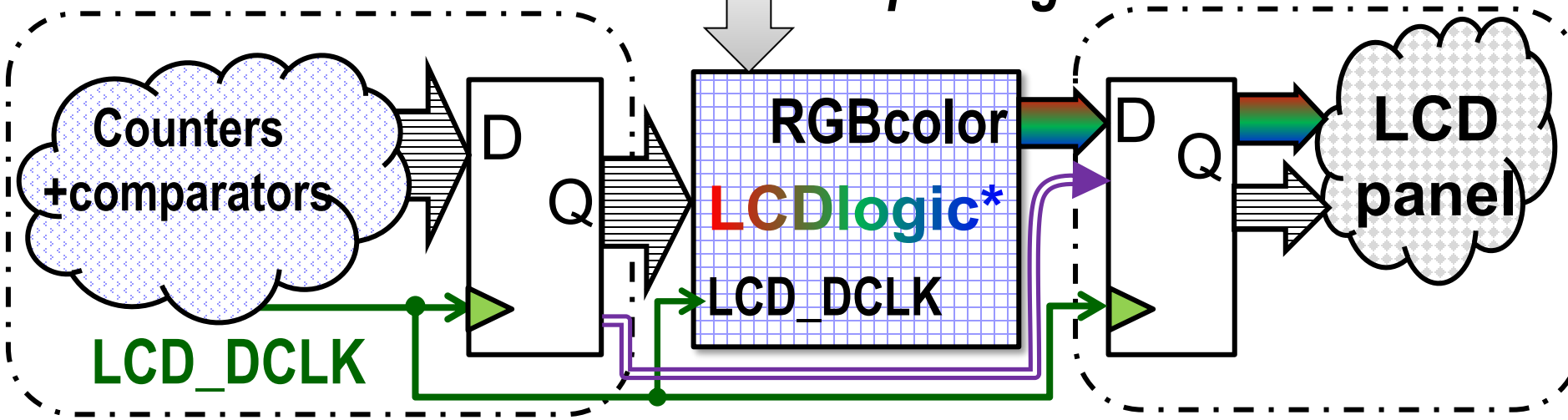
TOUCH_I2C_SDA
TOUCH_I2C_SCL
TOUCH_INT_N

I2C

iTouch : **entity** work.**VeekMT2_I2CTouchLCD**

Setting new values for the controls

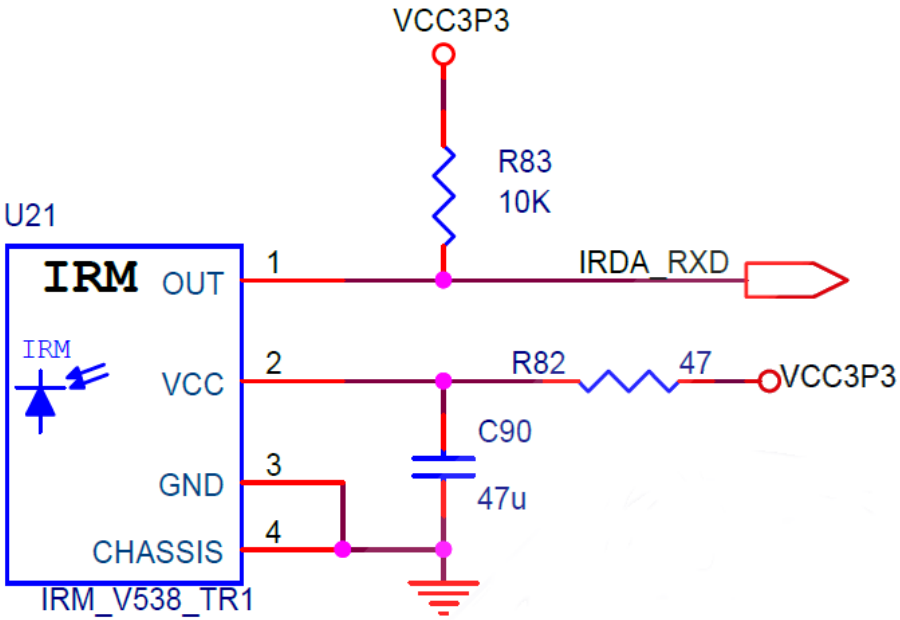
Outputting values



VeekMT2_LCDgenV2

VeekMT2_LCDregV2

IRDA_RDX



IRM-V538-TR1

Fig. 2 Example message when the key on the remote control remains pressed

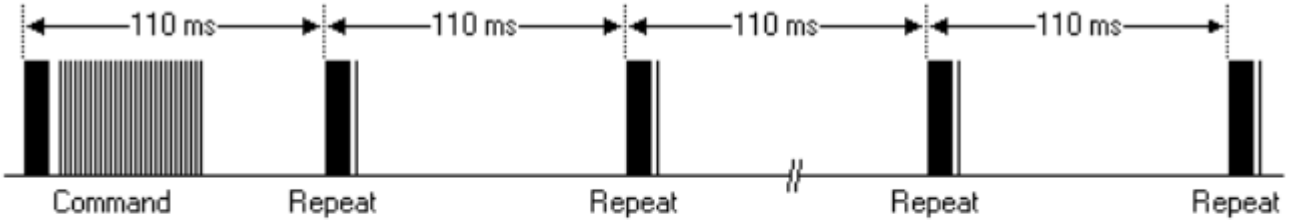
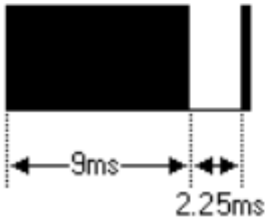


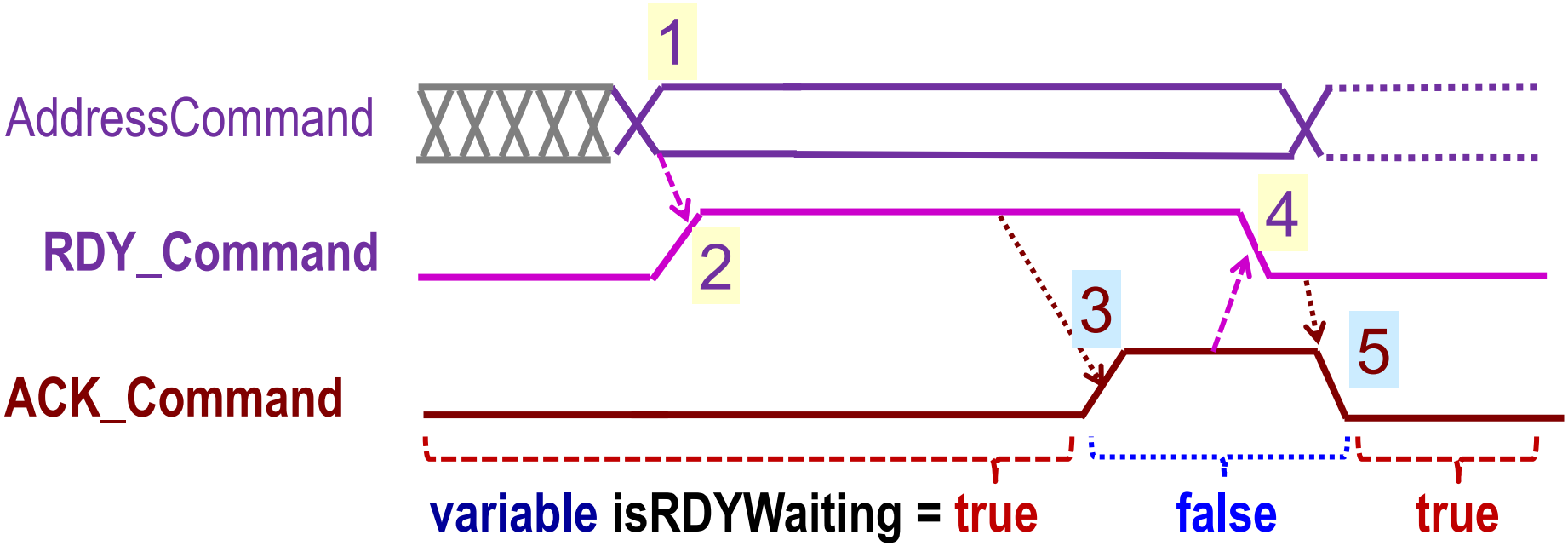
Fig. 3 One burst of code



Ad Notam: IR protocol (pdf)

```
entity VeekMT2_IRDAv2 is
  generic(IRDA_REPEAT_WAIT_MS : natural := 500);
```

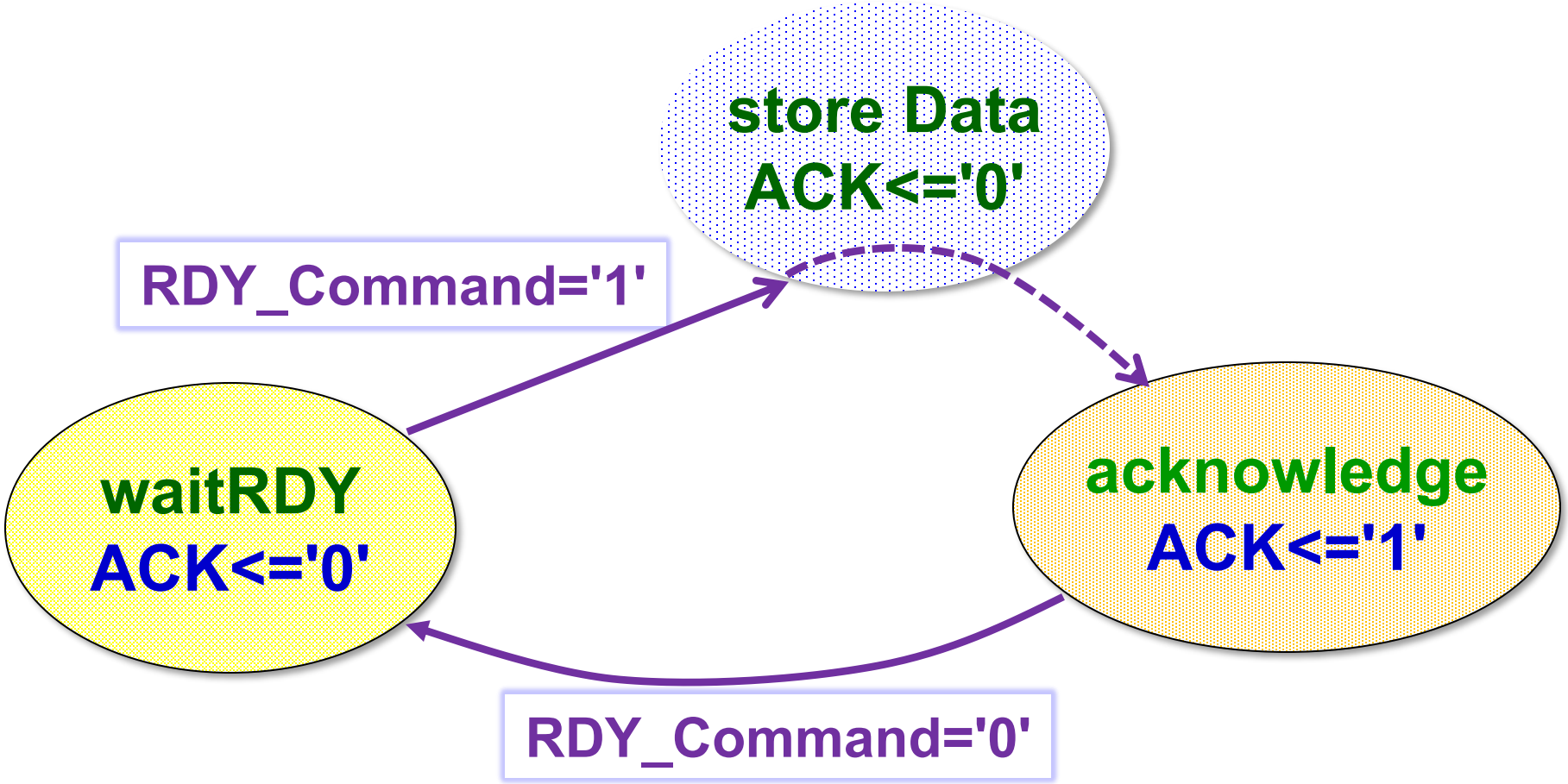
VeekMT2_IRDAv2 in UserInterface



similar to TCP = Transmission Control Protocol)

IRDA RDY-ACK FSM

FSM - Finite State Machine



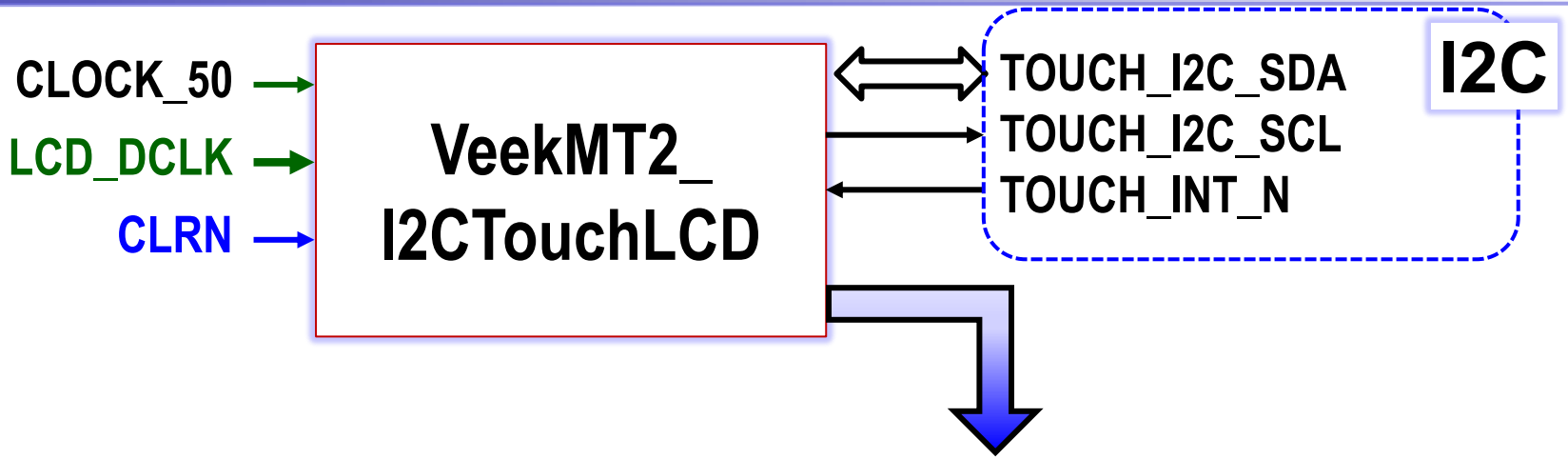
variable isRDYWaiting = true

false

IRDA RDY-ACK FSM

```
iFSM: case isRDYWaiting is -- for clarity, we used case instead of if-else
  when true=> -- we are waiting RDY
   iRdyCmd: if RDY_command='1' then -- we received RDY
            Command    := AddressCommand(7 downto 0); -- key code
            Repeat     := CommandIsRepeated; - storing flag repeated
            iCommand : case to_integer(unsigned(Command)) is
                       -- Play and on/off remote control buttons
                       when 16#16# | 16#12# => vStop := not vStop;
                       when others => null;  -- no command yet
                  end case iCommand;
          isRDYWaiting   := false; -- goto RDY acknowledge
      end if iRdyCmd;
   when false=>   ACK_Command <= '1';  -- we confirm receiving RDY
       if RDY_command='0' then  -- our ACK was received
          isRDYWaiting:=true; end if; -- goto the waiting for the next RDY
end case iFSM;
```

VeekMT2_I2CTouchLCD in UserInterface




```
subtype TouchDataSlv_t is std_logic_vector(42 downto 0);
```

```
function to_TouchRecord()…
```

```
type TouchRecord_t is
 record x1, y1, x2, y2 : integer range 0 to XCOLUMN_MAX;
        count : integer range 0 to 3;   -- 0 none
        gesture : integer range 0 to 7;
   -- 1 Move up, 2 Move left, 3 Move down, 4 Move Right, 5 Zoom In, 6 Zoom Out, 0 none
 end record;
```

Main elements in iReadTouchIrda process

```
iReadTouchIrda : process(LCD_DCLK,YEND_N)
 variable touchRecord : TouchRecord_t := TouchRecord_ZERO;
 variable isTouch, isTouchMem : boolean:=false;
 variable vStop  : std_logic:='0'; --  commandStop output
begin
    iRedge:if rising_edge(LCD_DCLK) then
```

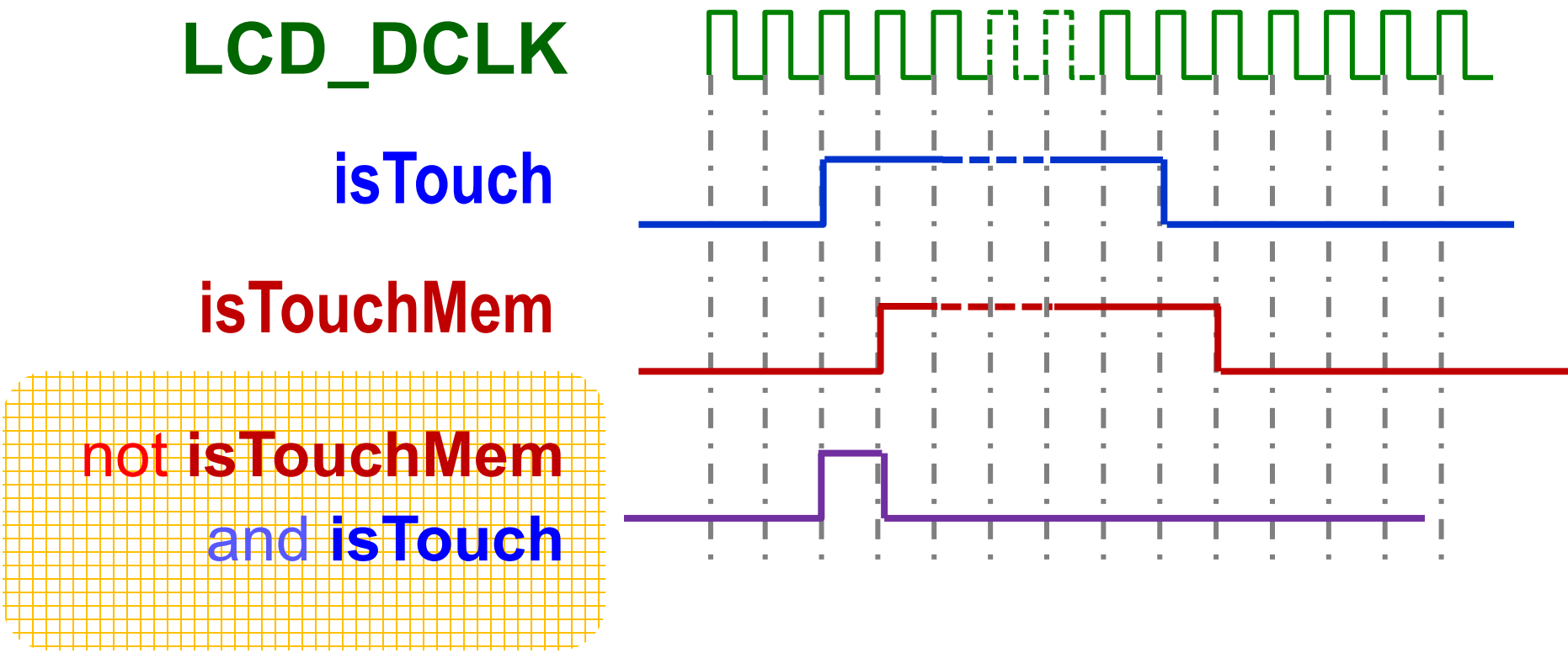
LCD touches

RDY-ACK handshake of IRDA

```
            end if iRedge;
     if falling_edge(YEND_N) then  -
        commandStop <=vStop; --it will be stable for the whole frame
    end if;
end process;
```

Detecting Touch Changes

without creating a dependency on two hours



The stop signal, wait, and speed values depend on **two inputs**: the commands from the infrared control and the touches. Finger movements are slower. Comparing them with their last states can detect the new touches.

iReadTouchIrda : process

```
iRedge:if rising_edge(LCD_DCLK) then
        touchRecord := to_TouchRecord(touchDataSlv);
        isTouch := touchRecord.Count > 0
                and inLimit(touchRecord.x1, 400-48, 96)
                and inLimit(touchRecord.y1, 240-48, 96);
    if(not isTouchMem and isTouch) then
            vStop:=not vStop;
    end if;
    isTouchMem:=isTouch;
```

-- these positions are also used in LCDlogic*

The code of IRDA RDY-ACK handshake FSM

```
    end if iRedge;
if falling_edge(YEND_N) then
    commandStop <=vStop; -- output to LCDLogicTask4
end if;
```

iReadTouchIrda : process

LCD Touches with the constants

```
iRedge:if rising_edge(LCD_DCLK) then
        touchRecord := to_TouchRecord(touchDataSlv);
        isTouch := touchRecord.Count > 0
                and inLimit(touchRecord.x1,
                BLINK_XLEFT-TCIRCLE, BLINK_SIZE+2*TCIRCLE )
                and inLimit(touchRecord.y1,
                BLINK_YTOP-TCIRCLE, BLINK_SIZE+2*TCIRCLE);
        if(not isTouchMem and isTouch) then
            vStop:=not vStop;
        end if;
        isTouchMem:=isTouch;
    end if iRedge;
```

Ulpack.vhd

The code of IRDA RDY-ACK handshake FSM

```
if falling_edge(YEND_N) then
    commandStop <=vStop; -- output to LCDLogicTask4
end if;
```

Constants in Process of LCDlogic*

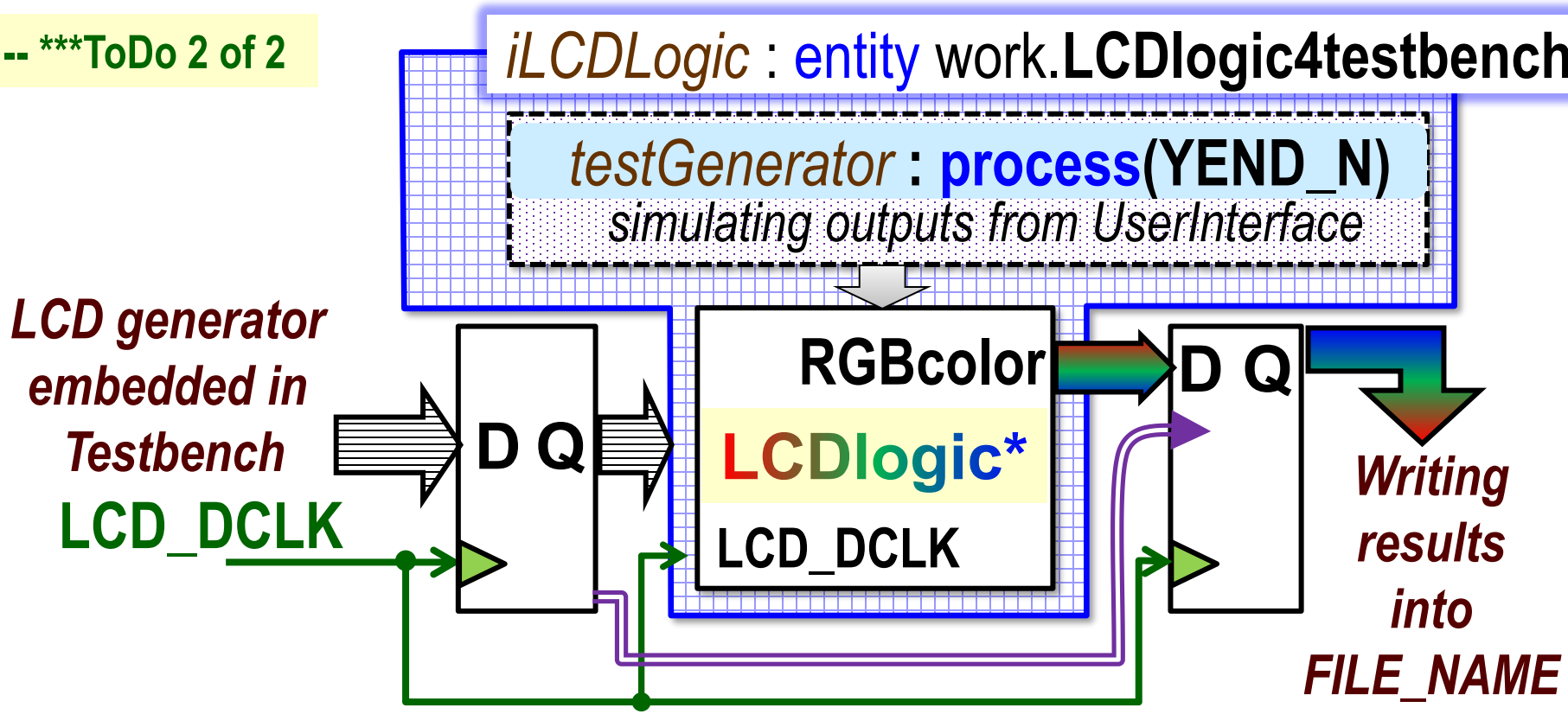
```
LSPimage : process(xcolumn, yrow, touchCoordinates, commandStop, blinkOnOff)
 variable RGB   : RGB_t     := BLACK;
 variable x, y  : integer   := 0; -- integer xcolumn and yrow
 variable touchRecord : TouchRecord_t  := TouchRecord_ZERO; -- in TouchIRDApackV2.vhd
 begin    ------------------------------------------------------------------------------------------------------
  x := to_integer(xcolumn);   y := to_integer(yrow);
  touchRecord := to_TouchRecord(touchCoordinates); -- defined in TouchIRDApackV2.vhd
  RGB := assignIf(IsTestbench, BLUE, NAVY); -- changing background color as version ID
  if InLimit(x,BLINK_XLEFT, BLINK_SIZE) and InLimit(y, BLINK_YTOP, BLINK_SIZE) then
    if commandStop then RGB:=GRAY;  else RGB:=assignIf(blinkOnOff, YELLOW, VIOLET);
    end if;
 end if;
  ----- draw touch circles-------------------------------------------------------
 if touchRecord.Count>0
    and (x-touchRecord.x1)**2+(y-touchRecord.y1)**2<TCIRCLE**2  then RGB:=RED; end if;
 if touchRecord.Count>1
    and (x-touchRecord.x2)**2+(y-touchRecord.y2)**2<TCIRCLE**2 then RGB:=GREEN; end if;
 RGBcolor <= RGB;
end process;  ------------------------------------------------------------------------------------------------------
```

testbenchV2_ControlPanel

-- ***ToDo 1 of 2

```
constant FILE_NAME : string := "C:/SPS/testbenchLCD.txt";
constant ENABLE_COMPRESSION : boolean := TRUE;
constant SINGLE_FRAME : boolean := FALSE;
constant COUNT_OF_FRAMES : positive := 32;
```

-- ***ToDo 2 of 2



Test Generator

```
testGenerator : process(YEND_N)
  variable tr:TouchRecord_t:=TouchRecord_ZERO;
  variable cntrSimStep:integer range 0 to 31;  -- for generating test signals
 begin
    if falling_edge(YEND_N) then -- YEND_N='0' in the last row of LCD frames
      -- At the beginning of the simulation, we emulate one touch only.
       tr.count:=1; tr.x1:=(cntrSimStep+1)*25; tr.y1:=(cntrSimStep+1)*15;
      -- In the middle, we add the second touch
       if cntrSimStep>16 then tr.count:=2; tr.x2:=tr.x1/2; tr.y2:=LCD_HEIGHT-tr.y1/2;  end if;
       -- We increment the simulation counter for 32 LCD frames
       if cntrSimStep<31 then cntrSimStep:=cntrSimStep+1; end if;
      -- We have copied the following expression from UserInterfaceV2.vhd
      if inLimit(tr.x1,BLINK_XLEFT-TCIRCLE, BLINK_SIZE+2*TCIRCLE  )
         and inLimit(tr.y1, BLINK_YTOP-TCIRCLE,  BLINK_SIZE+2*TCIRCLE ) then
           commandStop_s<='1';  else   commandStop_s<='0';
       end if;
     end if; -- if falling_edge(YEND_N)
    touchCoordinates_s<=to_TouchDataSlv(tr); -- pack TouchRecord_t tr to std_logic_vector
 end process;
```

More details

- Encoding of the NEC signal of the infrared remote control (IR remote) and its decoding

  Logic Circuits on FPGAs - version 3.2 and higher

  Chapter 7.7 page from 145 to 151

- RDY-ACK protocol - LSP lectures about FSMs